

Face Emotion Detection System – Detailed Code Explanation

This document explains the working of the Face Emotion Detection System implemented using Keras, OpenCV, and Python. The system performs two main tasks:

1. Training a Convolutional Neural Network (CNN) on an emotion image dataset
2. Performing real-time emotion detection from a webcam feed

1. Imports and Libraries

```
import os
import numpy as np
import cv2
import keras
from keras import layers, models
from keras.utils import to_categorical
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
```

- `os` is used for directory and file handling
- `numpy` is used for array and numerical operations
- `cv2` (OpenCV) is used for image processing and webcam access
- `keras` is used to build and train the CNN model
- `train_test_split` is used to divide dataset into training and testing sets

2. EmotionDetectionModel Class (Training Part)

This class handles dataset loading, model creation, training, and saving.

Initialization

```
self.model = None
self.emotions = ['Angry', 'Fear', 'Happy', 'Sad', 'Surprise']
self.img_size = 48
```

- Defines emotion labels
- All images resized to 48x48 pixels

CNN Model Architecture

The model consists of three convolutional blocks followed by dense layers.

- Conv2D layers extract facial features
- BatchNormalization stabilizes learning
- MaxPooling reduces feature size
- Dropout prevents overfitting
- Dense layers perform classification

The final layer uses softmax activation to output probabilities for each emotion.

Loading Dataset

The dataset must be arranged as:

```
dataset/  
  Angry/  
  Fear/  
  Happy/  
  Sad/  
  Surprise/
```

Steps:

- Read images in grayscale
- Resize to 48x48
- Normalize pixel values (0 to 1)
- Convert labels to one-hot encoding

Data Augmentation

For each image:

- Original image
- Horizontally flipped image
- Slightly rotated image

This increases training data and improves accuracy.

Training

- Split dataset into 80% training and 20% testing
- Train model using `model.fit()`
- Evaluate using test data

Saving and Loading Model

Model is saved in Keras format:

```
emotion_model.keras
```

3. RealtimeEmotionDetector Class (Webcam Part)

This class performs real-time emotion detection using the trained model.

Loading Model

```
self.model = keras.models.load_model(model_path)
```

Haar Cascade Classifiers

Used for detecting:

- Faces
- Eyes

These are pre-trained classifiers provided by OpenCV.

Preprocessing Face

Before prediction:

- Resize to 48x48
- Normalize pixel values
- Reshape for CNN input

Emotion Prediction

- Model predicts probabilities
- Highest probability emotion is selected
- Confidence score is displayed

Detect and Display

For every webcam frame:

1. Convert to grayscale
2. Detect faces
3. Detect eyes inside face

4. Predict emotion
5. Draw rectangle and display text

Webcam Loop

- Capture frames continuously
 - Process and display results
 - Press 'q' to quit
-

4. Main Function

Provides three options:

1. Train new model
 2. Run real-time detection
 3. Train and immediately run detection
-

5. Important Note (Bug)

Training uses 5 emotions:

```
['Angry', 'Fear', 'Happy', 'Sad', 'Surprise']
```

Realtime detection uses 6 emotions:

```
['Angry', 'Fear', 'Happy', 'Neutral', 'Sad', 'Surprise']
```

These must match, otherwise predictions will be incorrect.

6. Overall System Flow

```
Dataset Images -> CNN Training -> Saved Model
                                     -> Webcam -> Face Detection -> Emotion
Prediction
```

This project combines deep learning, computer vision, and real-time inference into a complete working system suitable for academic projects and demonstrations.